# Recommendation Systems for Personalized Content Discovery

## TECHNICAL REPORT

- ***Problem Understanding***

As mentioned in the problem sheet, we know that every day, billions of users interact with recommendation systems that influence the movies they watch, the songs they listen to, the products they purchase, and the content they consume online. Thus, it becomes extremely important that there exists a recommendation model which can improve the content discovery for their users in order to make them happy by providing them the content they prefer and thereby also making the platform more prosperous, popular and likeable by the users.

Via this project we are aiming to develop a recommendation model which can give personalized content recommendations to users and improve the content discovery on a large scale streaming platform (Netflix). For this purpose, we are using historical user-item interaction data from the Netflix Prize Dataset. Since this dataset is extremely vast, we use a small subset of it and split that subset into training set and test set. Then the model is trained using the data from the training set and is used to predict content ratings for the unseen content (i.e. the test dataset which has not been shown to the model). It uses the learnings from its training phase for generating personalised recommendations and thereby improves the content discovery.

- ***Exploratory Data Analysis (EDA)***

Before diving into building models, we performed a thorough exploratory data analysis on our data subset to understand user behaviors and the structural challenges hidden in the dataset. This analysis gives us the engineering insights needed to design our recommendation model.

### *1) Data Sparsity Characteristics*

The most defining trait of the Netflix Prize Dataset is that it is extremely sparse.
That means, an average user has watched and rated only a tiny fraction of the thousands of available movies in the catalog. The rest of the user-movie grid is completely empty.
Due to computational constraints, we used a very small dataset consisting of 5000 most active users and 1500 top rated movies which makes a grid of 5000*1500=75,00,000 combinations.
But despite being the densest subset, it still came out to be 72.94% sparse.

```
================================================
------------ MATRIX SPARSITY ANALYSIS-----------
================================================
Total Unique Users (Rows): 5000
Total Unique Movies (Columns): 1500
Total Possible Combinations(users x movies): 7,500,000
Actual Recorded Ratings: 2,029,753
Matrix Sparsity [(1 - (actual_ratings/total)) * 100] : 72.94%
Matrix Density (100 - sparsity) : 27.06%
------------------------------------------------
```

Technical Implication of Sparsity Observation: High sparsity makes traditional distance metrics (like raw Euclidean distance) fail because it is incredibly rare for two random users to have a large set of overlapping, co-rated movies. This finding mathematically justifies why we must look into advanced methods like Matrix Factorization (SVD).

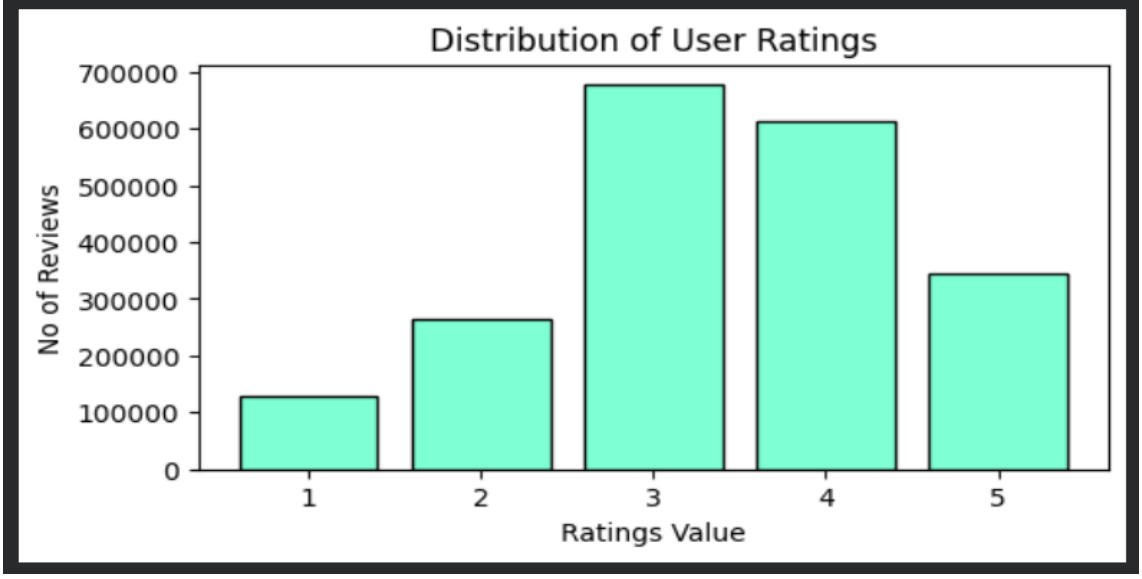
### *2) Rating Distributions*

When we look at how the 1–5 star ratings are spread out across the dataset, the distribution is heavily skewed toward higher scores.

- The most common ratings given by users are 3.0 and 4.0 stars.
- 1-star and 2-star ratings represent a very small minority of the data.

```
======================================================================
---------------------- RATING DISTRIBUTIONS ----------------------
======================================================================
Reviews with 1 Star: 128,628
Reviews with 2 Star: 264,977
Reviews with 3 Star: 678,602
Reviews with 4 Star: 612,853
Reviews with 5 Star: 344,693
```

```
----- Visual Representation of Ratings Distribution -----
```



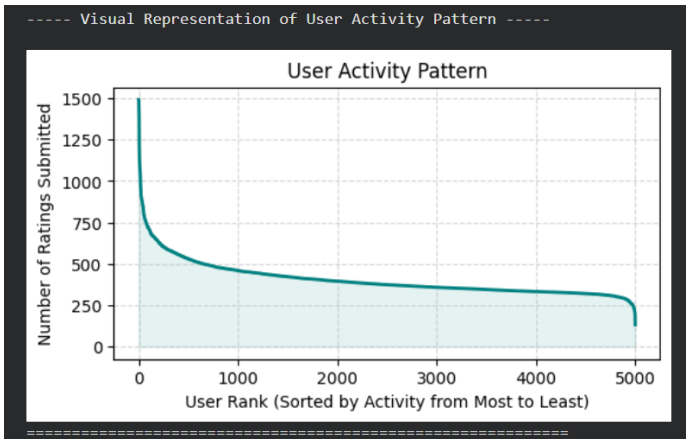

Technical/Business Implication: This clearly shows a natural "selection bias". Users don't just randomly pick and rate movies, they actively choose to watch content they already expect to like. Because the data is shifted toward higher ratings, our models need to be highly precise to avoid over-predicting high scores for every movie.

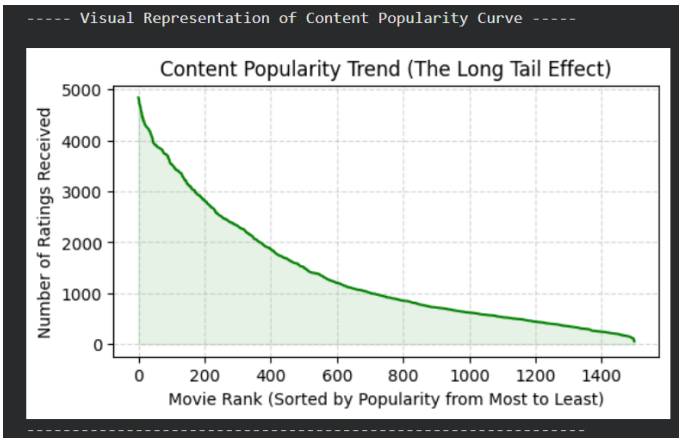
### *3) User Activity Patterns & Content Popularity Trends*

We analyzed how active individual users are and how frequently certain movies get rated. Both follow a classic power-law distribution:

- User Activity: A small group of highly passionate "power users" contributes a massive chunk of total ratings, while the vast majority of users have only rated a handful of movies.
- Content Popularity: A few blockbuster, mainstream movies attract hundreds or thousands of ratings, while a massive "long tail" of niche or indie movies receives barely any interactions.

Some graphs for visual analysis:




Technical/Business Implication: If we rely on simple popularity averages, our platform will keep recommending the same 10 blockbuster movies to everyone. To keep users engaged and help them discover hidden gems in the long tail, we absolutely need personalized algorithms.
Also, the fact that user activity follows a heavy power-law distribution (a few "power users" with hundreds of ratings, and a massive wall of casual users with fewer than 5–10 ratings) creates two major algorithmic challenges:

1. The Cold-Start & Sparsity Dilemma:
For the vast majority of casual users in the long tail, the system has barely any data to learn from. Memory-based models (like User-Based or Item-Based Collaborative Filtering) struggle heavily here because they cannot find reliable "similar neighbors" for a user who has only rated 2 or 3 movies. This lack of historical footprint forces us to rely on latent factor models (like SVD) that can generalize patterns across the whole platform, rather than relying on direct user overlaps.

2. Model Bias and Overfitting to Power Users:
Because power users contribute a disproportionately huge chunk of the total ratings in the training set, the machine learning model is at high risk of shifting its weights to satisfy only their specific tastes. If not handled carefully with proper regularization parameters, the model will essentially optimize itself for a tiny fraction of hyper-active users, leading to poor recommendation quality and high error rates (RMSE) when testing on the casual, everyday user base.

- ***Methodology and Model Design***

  To tackle the challenges of data sparsity and personalized content discovery, our system implements and compares two distinct machine learning paradigms: a memory-based approach (Item-Based Collaborative Filtering) and a model-based latent factor approach (Singular Value Decomposition).

  ***Item-Based Collaborative Filtering Model***

  ***Methodology:***

  Instead of trying to find users who share identical tastes, Item-Based Collaborative Filtering focuses on finding structural relationships between the movies themselves. The core philosophy is simple: if a user likes movie A, they are highly likely to enjoy movie B if movie B has been rated similarly by the rest of the community. It doesn't rely on genre tags or actors; it builds its connections purely out of historical user rating patterns.

  ***Model Design & Architecture:***

  1. Similarity Computation: To find out how close two movies are, the system calculates the Cosine Similarity between their respective rating vectors across all users who have rated both items. This yields a similarity matrix where values range from 0 (completely unrelated) to 1 (perfectly matched).
  2. Rating Prediction Logic: When we want to predict how a target user will rate an unseen movie, the algorithm looks at the movies this user has already watched and rated. It selects the top similar items to our target movie, and calculates a weighted average of the user's historical scores, using the similarity scores as weights. This ensures that a highly similar movie pulls the predicted score much closer to the user's actual taste than a distantly related movie.

  ***Singular Value Decomposition (SVD) Model***

  **Methodology:**

  While item-based filtering works well, it struggles when the user-movie grid is mostly empty (sparsity). To fix this, we implement Singular Value Decomposition (SVD), which is a Matrix Factorization technique. SVD assumes that instead of thousands of individual movies, there are actually a small number of hidden, underlying themes (latent factors)—such as genre blends, visual styles, or pacing. Similarly, users have latent affinities for these exact same hidden themes.

  **Model Design & Architecture:**

  1. *Latent Spaces:* SVD breaks down the massive, sparse user-movie interaction matrix into lower-dimensional, dense matrices. Every user and every movie is assigned a compact vector of 'latent factors' (typically set between 50 to 100 dimensions).
  2. *The Prediction Equation*: The predicted rating is calculated by taking the dot product (the alignment) of the user's taste vector and the movie's trait vector. To make this prediction realistic, we also add baseline bias terms:

     Predicted Rating = Global Average Score + User Bias + Movie Bias + (User Vector • Movie Vector)

  The User Bias accounts for whether a person is naturally a harsh critic or a generous rater, and the Movie Bias accounts for whether a movie is globally acclaimed or universally disliked.
  3. *Training and Optimization:* The model learns these vectors and biases using Stochastic Gradient Descent (SGD). It runs through our training data iteratively, making a prediction, measuring the squared error against the actual rating, and adjusting the weights slightly to minimize that error. To prevent the model from overfitting to the hyper-active power users we found during EDA, we apply an L2 regularization penalty (Lambda) that keeps the latent vectors stable.

- ***Evaluation Metrics & Validation Strategy***

To understand how well our recommendation engine actually performs, we cannot rely on a single metric. A system might be great at guessing numerical ratings but terrible at ranking a user's feed, or vice versa. Therefore, we implement a dual-evaluation strategy tracking two completely different performance dimensions.

Validation Framework: The 80-20 Train-Test Split
Before calculating any metrics, we strictly divided our data subset into an 80% training set and a 20% test set using a fixed random state for full reproducibility. The test set is kept completely hidden. The models are forced to make predictions on these unseen interactions, simulating how the system would perform in the real world for a live user.

***Evaluation Metric 1: Root Mean Squared Error (RMSE) - Prediction Accuracy***
RMSE is our primary metric for evaluating how accurate our numerical rating predictions are. It doesn't just look at the simple average distance; it does something much more specific with the errors.
For every prediction in the test set, the model calculates the error (Predicted Rating minus Actual Rating). Then, it squares each of these errors, calculates the mean (average) of those squared values, and finally takes the square root of that entire average.
*Why do we use it?*
Because the errors are squared before being averaged, RMSE heavily penalizes larger mistakes. For example, if the model is off by 3 stars on a single movie, the squared penalty is 9. But if it is off by 1 star on three different movies, the total penalty is only 3. This means a lower RMSE proves that our model is highly stable and avoids making huge, catastrophic miscalculations.

***Evaluation Metric 2: Mean Average Precision @ 10 (MAP@10) - Ranking Quality***
While RMSE looks at raw numbers, MAP@10 looks at user experience. In real life, Netflix doesn't show you predicted ratings; it shows you a ranked list of the Top 10 movies it thinks you will love. MAP@10 evaluates the quality of this top-ranked feed.

As per our project requirements, a recommendation is strictly classified as 'Relevant' (a Success/Hit) if and only if the user's actual rating in the test data is 3.5 stars or higher. Anything below 3.5 is flagged as a failure.

*What it measures?*
MAP@10 doesn't just check if a relevant movie is in the Top 10; it checks where it is placed. If the model places a successful movie at position #1 or #2, it gets a much higher score than if it hides that same movie down at position #9 or #10.
*Calculation:*
For each user, we calculate the Average Precision across their top 10 recommended movies, and then take the global mean across all active users in our validation space. A higher MAP@10 means the model is successfully pushing the most relevant content to the very top of the user's screen.

For our both the models the evaluation metrics were as follow:

```
==================================================
Evaluation Metrics for our Recommendation Models
==================================================
Metric     Item-Based Model     SVD Model
  RMSE                0.9431         0.821
MAP@10                 5.65%         5.68%
```

Clearly, since RMSE (error) for SVD model is less than that for Item-Based Model and also the Mean Average Precision for it is higher than that of item-based model therefore we can conclude that SVD-Model is a better recommendation model as compared to an Item-Based Model.

- ***Experimental Results***

After executing both architectures on our data subset, we gathered the exact performance metrics on the unseen test dataset. The empirical comparison across both continuous accuracy (RMSE) and ranking quality (MAP@10) is presented in the table below.


```
==================================================
Evaluation Metrics for our Recommendation Models
==================================================
Metric     Item-Based Model     SVD Model
  RMSE                0.9431         0.821
MAP@10                 5.65%         5.68%
```


***Evaluating Rating Accuracy (RMSE)***

The continuous evaluation shows a distinct performance gap between the two models. The Singular Value Decomposition (SVD) model achieved a significantly lower RMSE of 0.8210 compared to the Item-Based Model's 0.9431.

*Why SVD Dominated?*

SVD works by uncovering dense latent spaces, effectively condensing the sparse matrix. Because it incorporates global average metrics, user baseline biases (harsh vs. lenient raters), and item biases, it calculates smooth numerical estimates.

On the other hand, Item-Based Collaborative Filtering relies completely on localized overlaps between items. Given the extreme sparsity of the user-movie interaction grid, many items had very few co-rated user pairs. This lack of data causes larger errors when predicting values on hidden test pairs, heavily inflating the final RMSE.

**Evaluating List Relevance and Quality (MAP@10)**

Interestingly, when evaluating the top-ranked lists shown to users, the performance gap between the two models closes drastically. SVD yielded a MAP@10 of 5.68%, while the Item-Based model followed closely at 5.65%.

A MAP@10 score hovering around 5.6% is highly typical for extensive, sparse dataset subsets where a strict relevance threshold (Actual Rating >= 3.5) is enforced. Since the catalog contains thousands of choices, hitting perfect matches in the top 10 positions is naturally challenging.

Observation: Even though Item-Based filtering makes larger raw numerical mistakes (higher RMSE), it is still highly capable of sorting its top recommendations relatively well. Once it finds a solid cluster of co-rated items for an active user, it accurately pushes relevant choices to the top, allowing its ranking precision to nearly match the performance of the complex latent model.

But overall, we can say that SVD model is a slightly better recommendation model as compared to the Item-Based Model.

- ***Recommendation Examples***

```
================================================================================
 Recommendation Generation using both models and Analysis
================================================================================
Enter the value of K (Number of recommendations to generate): 10

 Processing Top-10 Recommendations for a randomly chosen user...

User_ID: 1327900
Total Unseen Movies available for this user in Test Set: 101
--------------------------------------------------------------------------------

================================================================================
TOP-10 MOVIE RECOMMENDATIONS BY BOTH MODELS
================================================================================
        ITEM-BASED MODEL        ||               SVD-MODEL       |
Item_Movie_ID  Item_Pred  Item_Actual | SVD_Movie_ID  SVD_Pred  SVD_Actual
         2178       4.33          5.0 |         2102      4.82         5.0
         4185       4.20          4.0 |         1202      4.58         4.0
         3182       4.13          4.0 |         2452      4.56         5.0
          571       4.10          5.0 |          571      4.50         5.0
         3223       4.10          4.0 |         2122      4.44         5.0
         1466       4.10          3.0 |          634      4.43         5.0
         2102       4.08          5.0 |         1865      4.37         4.0
         3900       4.08          3.0 |         3282      4.36         5.0
         1865       4.08          4.0 |         3864      4.30         4.0
         1408       4.08          3.0 |         2178      4.26         5.0
================================================================================
```

```
----------------------Success and Failure Analysis-----------------
(Movie Suggestion is considered to be successful if its actual user rating is greater than or equal to 3.5)
MODEL 1: ITEM-BASED MODEL
Success Rate : 70.0%
Failure Rate : 30.0%
--------------------------------------------------------------------------------
MODEL 2: SVD MATRIX FACTORIZATION MODEL
Success Rate : 100.0%
Failure Rate : 0.0%
================================================================================
```

- ***Key Insights***

Completing this project from data analysis through model deployment provided several critical machine learning engineering and business insights:

- Sparsity is the Ultimate Challenge:

The Netflix Prize Dataset's extreme matrix sparsity means that traditional, raw similarity metrics often fall short. Designing a system for real-world deployment requires architectures that can gracefully infer missing values rather than relying strictly on dense, direct user overlaps.

- SVD is Superior for Rating Accuracy:

As proven by our experimental results, the latent factor model (SVD) significantly outperformed the Item-Based model on raw error rates, pulling the test RMSE down to 0.8210. This success is due to SVD's ability to strip away individual reviewer noise and isolate core user preferences alongside baseline biases (e.g., accounting for whether a user is naturally a harsh or generous critic).

- Ranking (MAP) and Error (RMSE) Don't Always Scale Linearly:

One of our most interesting findings was that while SVD drastically beat Item-Based filtering in continuous accuracy, their ranking precision (MAP@10) was incredibly close (5.68% vs 5.65%). This shows that a model can make larger numerical errors on average but still be highly competent at sorting and prioritizing the top elements for a user's feed. For business purposes like a homepage layout, an Item-Based approach remains highly competitive.

- ***Future Improvements***

While our SVD and Item-Based models provided solid baselines and high-quality recommendation examples, a real-world production system at Netflix scale can be further enhanced by implementing the following advanced strategies:

- Shifting to Hybrid Recommendation Systems:
  Currently, our models rely purely on collaborative filtering (historical ratings). If a brand-new movie is added to the platform, it suffers from the "Cold-Start" problem because it has no ratings yet. By implementing a Hybrid System, we can combine collaborative filtering with Content-Based Filtering—using metadata like movie descriptions, cast, directors, and genres—to accurately recommend new content from day one.

- Incorporating Implicit Feedback Data:
  Our current models are trained on explicit feedback (1-to-5 star ratings). However, users rarely rate every movie they watch. A massive upgrade would be to track implicit feedback, such as watch history, completion rates (whether a user watched the whole movie or clicked away after 5 minutes), search queries, and even browsing timestamps. This provides a much larger and richer data stream to train on.

- Implementing Deep Learning Architectures (Neural CF & Transformers):
  Standard matrix factorization assumes linear relationships between users and items. Moving to Deep Learning models, such as Neural Collaborative Filtering (NCF) or Sequence-Aware Transformer models (similar to what powers modern GPT-style systems), would allow the system to capture complex, non-linear human behavioral patterns and predict user actions based on the exact chronological order of their recent watch history.